

Travaux dirigés n° 7
Décoration de grammaire

Exercice 1 (Décoration de grammaire - 1)

Nous souhaitons pouvoir créer un programme permettant de calculer et afficher des opérations entières ou réelles. La grammaire est la suivante :

$$\begin{aligned} L &\rightarrow E \text{'\n'} L \\ L &\rightarrow \epsilon \\ E &\rightarrow E_1 + T \mid T \\ T &\rightarrow T_1 * F \mid F \\ F &\rightarrow entier \mid reel \mid (E) \end{aligned}$$

Nous considérons les fonctions suivantes :

```
/**
 * Affiche une valeur à l'écran.
 * @param valeur la valeur à afficher (entier ou réel)
 */
afficher(valeur)

/**
 * Convertit une valeur dans un autre type.
 * @param valeur la valeur à convertir
 * @param type INT pour convertir un réel en entier et DOUBLE pour l'inverse
 * @return la valeur
 */
convertir(valeur, type) : valeur

/**
 * Convertit une chaîne de caractères dans un type donné.
 * @param str la chaîne à convertir
 * @param type INT pour convertir en entier et DOUBLE pour convertir en réel
 * @return la valeur
 */
convstr(str, type) : valeur
```

1°) Quels sont les attributs nécessaires pour chaque symbole de la grammaire ? Précisez s'ils sont synthétisés ou hérités.

2°) Proposez une décoration de la grammaire permettant d'afficher le résultat des opérations dès que l'utilisateur appuie sur la touche «entrée» (on ne gère pas les erreurs).

Exercice 2 (Décoration de grammaire - 2)

Nous souhaitons pouvoir créer des arbres d'évaluation typés (des arbres syntaxiques représentant une expression arithmétique) à partir de la grammaire suivante :

$$\begin{aligned} E &\rightarrow E_1 + T \\ E &\rightarrow T \\ T &\rightarrow T_1 * F \\ T &\rightarrow F \\ F &\rightarrow id \mid entier \mid reel \\ F &\rightarrow (E) \end{aligned}$$

Nous considérons les fonctions suivantes (la table des symboles est considérée existante et accessible à tout endroit ; elle constituée d'éléments de type `entree_t`) :

```

/**
 * Création d'une feuille.
 * @param type ID ou CST
 * @param valeur un entier, un réel ou un entree_t
 * @return un noeud
 */
creer_feuille(type, valeur) : noeud_t

/**
 * Création d'un noeud.
 * @param type opération ADD ou MUL ou conversion CONV_INT ou CONV_DOUBLE
 * @param noeud1 le premier argument
 * @param noeud2 le deuxième argument (NULL pour CONV_INT et CONV_DOUBLE)
 * @return un noeud
 */
creer_noeud(type, noeud1, noeud2) : noeud_t

/**
 * Retourne le type d'une entrée de la table des symboles.
 * @param str le nom de l'entrée
 * @return le type de l'entrée (constante INT ou DOUBLE)
 */
type_entree(str) : int

/**
 * Retourne l'entrée de la table des symboles.
 * @param str le nom de l'entrée
 * @return l'entrée
 */
entree(str) : entree_t

```

1°) Quels sont les attributs nécessaires pour chaque symbole de la grammaire ?

2°) Proposez une décoration de la grammaire permettant de créer les arbres d'évaluation.

Exercice 3 (Calculatrice en listes)

Nous souhaitons réaliser une calculatrice basée sur les listes non linéaires. Toute opération consiste en une liste ou une valeur. Le premier argument des listes est un opérateur (ici '+' ou '*'), les autres éléments pouvant être des listes ou des valeurs. Les listes vides ne sont pas reconnues.

Exemple d'exécution :

```

1
=> 1
(+, 2, 3)
=> 5
(*, 2.5, (+, 4, (*, (+, 1, 2), 2)))
=> 25

```

1°) Donnez la grammaire permettant de reconnaître ce langage.

2°) Quelles structures sont nécessaires pour la décoration de la grammaire ? Quelles fonctions ?

3°) Donnez les attributs nécessaires pour chaque symbole de votre grammaire.

4°) Donnez la décoration de la grammaire.